

Contents

Neue Games zu Capyt hinzufügen	1
1. Überblick	1
2. Ordnerstruktur	2
3. Neues Game anlegen	2
4. games.json	3
5. Game Lifecycle	5
6. Handshake und API-Version	5
7. Öffentliche Game-API	6
8. Events und Nachrichten	7
9. Resultate, Highscores und Run-ID	8
10. Coins und Reward Validation	8
11. Finanz-App und Datenschutzgrenze	9
12. Game Storage und Persistenz	9
13. Theme	9
14. Offline/PWA	10
15. Minimales vollständiges Beispiel	10
16. Tests und Checkliste	11
17. Referenzimplementierungen	12

Neue Games zu Capyt hinzufügen

Capyt 2.2.7a - Capyt Game API 1

Dieses Dokument beschreibt die tatsächlich in Capyt 2.2.7a implementierte Minigame-Schnittstelle. Ein Minigame ist ein plugin-artiges, isoliertes Web-Projekt unter `capy/games/projects/`. Es darf weder den Capy-State noch Coins, FP1-Daten oder Finanzdaten direkt verändern.

1. Überblick

Was ist der Game-Hub?

Der Button **Spiele** in der Capy-Bottom-Navigation öffnet den Minigame-Hub als normales Capy-Bottom-Sheet. Der Hub liest seine Einträge aus `capy/games/games.json`, sortiert sie nach order und zeigt nur Games mit `enabled: true`.

Statuswerte:

- `available`: startbar
- `coming_soon`: sichtbar, aber nicht startbar
- `experimental`: startbar und im Hub als Alpha markiert
- `disabled`: nicht als normales Production-Game gedacht

Wie funktioniert die Game Registry?

`games.json` ist die zentrale Registry. Name, Beschreibung, Status, Entry-Point, Reihenfolge, Reward-Limits, Capabilities und Offline-Dateien werden dort definiert. Ein neues Game soll deshalb im Normalfall nur einen eigenen Projektordner und einen Registry-Eintrag benötigen.

Wichtige Validierungen in Capyt Game API 1:

- Game-IDs müssen aus Kleinbuchstaben, Ziffern und Bindestrichen bestehen.
- Doppelte Game-IDs werden abgelehnt.
- Unbekannte Statuswerte werden abgelehnt.
- Startbare Entry-Points müssen lokal unter `./projects/<game-id>/index.html` liegen.
- Externe URLs, absolute Pfade und `../` Path Traversal werden abgelehnt.
- Nicht freigeschaltete oder unbekannte Capabilities werden abgelehnt.

Wie funktioniert die Game Bridge?

Ein Game läuft in einem iframe mit `sandbox="allow-scripts"`. `allow-same-origin` ist absichtlich nicht gesetzt. Das Game hat dadurch keinen normalen direkten Zugriff auf den Storage oder JavaScript-Kontext der Hauptanwendung.

Die Kommunikation erfolgt über `postMessage`. Games verwenden dafür nicht direkt `postMessage`, sondern das zentrale SDK:

```
<script src="../../js/game-sdk.js"></script>
```

Danach steht global `CapytGame` zur Verfügung.

Wie sind Games isoliert?

Ein Game darf nicht:

- `localStorage` der Hauptanwendung verändern
- `Capy-State` direkt setzen
- `Coins` direkt setzen
- Finanztransaktionen erzeugen
- `FP1-Payloads` verändern
- Storage anderer Games lesen oder überschreiben
- interne Module aus `capy/js/` oder der Finanz-App importieren

Persistente Game-Daten laufen ausschließlich über die Game Bridge.

2. Ordnerstruktur

```
capy/games/  
  games.json  
  css/  
    games.css  
  js/  
    game-hub.js  
    game-loader.js  
    game-bridge.js  
    game-rewards.js  
    game-storage.js  
    game-sdk.js  
  projects/  
    _template/  
      index.html  
      game.js  
      game.css  
      assets/  
      README.md  
      carrot-catch/  
      ...  
  docs/  
    NEW_GAME_GUIDE.md  
    Neue_Games_hinzufuegen.pdf
```

Jedes Game bleibt in seinem eigenen Projektordner. Es darf keine Dateien eines anderen Games voraussetzen oder verändern.

3. Neues Game anlegen

1. `capy/games/projects/_template/` kopieren.
2. Den neuen Ordner nach der stabilen Game-ID benennen, zum Beispiel `mein-game`.
3. `index.html`, `game.js`, `game.css` und eigene Assets anpassen.
4. Einen Eintrag in `capy/games/games.json` ergänzen.
5. Nur die wirklich benötigten Capabilities eintragen.
6. Reward- und Result-Validierung konfigurieren.

7. Game über `CapytGame.ready()` initialisieren und über `CapytGame.start()` starten.
8. Resultate nur über `submitScore()` und `finish()` senden.
9. `npm test` und `npm run check` ausführen.
10. PWA/Offline-Modus testen.

Das Template selbst wird nicht in `games.json` registriert und erscheint daher nicht im Hub.

4. games.json

Die Registry hat aktuell `schemaVersion: 1` und `apiVersion: 1`.

id

Stabile technische Game-ID, zum Beispiel:

```
"id": "carrot-catch"
```

Erlaubt sind Kleinbuchstaben, Ziffern und Bindestriche. Die ID bestimmt auch das Storage-Namespace.

name

Sichtbarer Name im Hub.

description

Kurze Beschreibung für die Game-Karte.

status

Einer von `available`, `coming_soon`, `disabled`, `experimental`.

enabled

Bei `false` wird das Game im normalen Hub ausgeblendet. Ein deaktiviertes internes Demo kann trotzdem als Referenz im Repository liegen.

version

Game-eigene Versionsnummer. Sie ist unabhängig von der Capyt-App-Version.

entry

Startdatei eines startbaren Games. In API 1 ist nur dieses Muster erlaubt:

```
"entry": "../projects/mein-game/index.html"
```

Externe URLs und Path Traversal sind nicht erlaubt.

icon

Kurzes Icon für die Hub-Karte. Alternativ kann später `image` verwendet werden.

image

Optionaler Bildpfad. In 2.2.7a nutzt der Hub primär `icon`.

orientation

`portrait`, `landscape` oder `any`. Capy bleibt Mobile-first und `portrait` ist die Standardausrichtung.

order

Numerische Sortierung im Hub. Kleinere Werte stehen weiter oben.

tags

Freie, kurze Metadaten wie arcade, quick oder skill.

rewards

Beispiel:

```
{
  "enabled": true,
  "currency": "coins",
  "maxCoinsPerRun": 15,
  "dailyRewardLimit": 75,
  "cooldownMs": 0,
  "strategy": {
    "type": "score",
    "scorePerCoin": 2
  }
}
```

scorePerCoin: 2 bedeutet: zwei Score-Punkte ergeben einen Coin, bevor die Limits angewendet werden.

In API 1 werden Rewards immer vom Host berechnet. maxCoinsPerRun, dailyRewardLimit und cooldownMs werden zentral berücksichtigt. Das Game kann keinen Coin-Betrag vorgeben.

resultValidation

Plausibilitätsgrenzen für Resultate:

```
{
  "minScore": 0,
  "maxScore": 250,
  "minDurationMs": 1000,
  "maxDurationMs": 120000
}
```

Ein Resultat außerhalb dieser Grenzen wird abgelehnt.

capabilities

Aktiv in Capyt Game API 1:

- capy.read
- game.storage
- game.score
- coins.reward
- theme.read
- app.read

Für spätere API-Versionen reserviert, aber in API 1 noch nicht freigeschaltet:

- capy.effect
- inventory.read
- inventory.reward
- finance.read.summary

Diese reservierten Capabilities dürfen in 2.2.7a noch nicht in einem aktiven Game angefordert werden. Die Registry-Validierung lehnt sie ab.

offlineAssets

Zusätzliche Dateien des Games, die bei der Service-Worker-Installation für aktivierte available/experimental Games automatisch gecacht werden:

```
"offlineAssets": [  
  "./projects/mein-game/game.js",  
  "./projects/mein-game/game.css",  
  "./projects/mein-game/assets/sprite.png"  
]
```

Die Pfade müssen im eigenen Projektordner bleiben. Der entry wird automatisch zusätzlich gecacht.

5. Game Lifecycle

Der normale Ablauf ist:

```
Game Hub  
-> Game Loader  
-> iframe / loading  
-> CapytGame.ready()  
-> Host prüft Game-ID, Session und Capabilities  
-> capyt.game.init  
-> CapytGame.start()  
-> playing  
-> submitScore() optional  
-> finish()  
-> Host validiert Resultat  
-> Host speichert Highscore und Run  
-> Host berechnet erlaubten Coin-Reward  
-> gemeinsamer Result-Screen  
-> erneut spielen oder zurück zum Hub
```

Interne Session-Zustände berücksichtigen loading, ready, playing, paused, finished und error.

6. Handshake und API-Version

Capyt Game API 1 führt beim Start einen Handshake durch. CapytGame.ready() sendet eine capyt.game.ready Nachricht. Der Host prüft aktive Session, gameId und die Registry-Konfiguration und antwortet mit capyt.game.init.

Beispiel für die Daten von ready():

```
{  
  "apiVersion": 1,  
  "gameId": "carrot-catch",  
  "sessionId": "game_session_...",  
  "runId": "game_run_...",  
  "theme": "dark",  
  "capabilities": ["capy.read", "game.score"],  
  "capy": {  
    "name": "NapoeIn",  
    "gender": "männlich",  
    "hunger": 74,  
    "happiness": 81,  
    "energy": 62,  
    "affection": 12,  
    "sleeping": false  
  }  
}
```

sessionId und runId werden vom Host erzeugt. Ein Game darf sie nicht selbst erfinden oder austauschen.

7. Öffentliche Game-API

await CapytGame.ready()

Stellt den Handshake her und liefert die Init-Daten. Mehrfache Aufrufe verwenden danach die bereits empfangenen Init-Daten.

```
const init = await CapytGame.ready();
```

await CapytGame.start()

Wechselt die aktive Session von ready nach playing.

```
await CapytGame.start();
```

await CapytGame.getCapy()

Benötigt capy.read. Liefert nur die freigegebene, read-only Capy-Zusammenfassung:

```
const capy = await CapytGame.getCapy();  
console.log(capy.name, capy.energy);
```

Nicht enthalten sind Coins, Inventar, kompletter Capy-State, Finanzdaten oder FP1-Daten.

await CapytGame.getAppInfo()

Benötigt app.read. Liefert:

```
version  
platform  
locale  
online  
pwa
```

Kontostände, Budgets, Buchungen und andere Finanzdaten werden nicht geliefert.

await CapytGame.getTheme()

Benötigt theme.read. Liefert light oder dark.

```
const theme = await CapytGame.getTheme();
```

CapytGame.onThemeChange(listener)

Registriert einen Listener für Theme-Wechsel während eines geöffneten Games. Das SDK setzt zusätzlich `document.documentElement.dataset.theme` automatisch.

```
const unsubscribe = CapytGame.onThemeChange(theme => {  
  console.log('Theme:', theme);  
});
```

CapytGame.onLifecycle(listener)

Empfängt Host-Lifecycle-Ereignisse wie paused, playing oder closed.

await CapytGame.storage.get(key)

Benötigt game.storage. Liest einen Wert aus dem eigenen Game-Namespace.

```
const highscore = await CapytGame.storage.get('highscore');
```

await CapytGame.storage.set(key, value)

Speichert JSON-kompatible Daten im eigenen Namespace.

```
await CapytGame.storage.set('difficulty', 'normal');
```

Ein einzelner Wert ist auf 16 KiB JSON-Daten begrenzt. Storage-Keys dürfen Buchstaben, Ziffern, Punkt, Unterstrich und Bindestrich enthalten und maximal 80 Zeichen lang sein.

await CapytGame.storage.remove(key)

Entfernt einen Key aus dem eigenen Game-Namespace.

await CapytGame.getGameData(key) / setGameData(key, value)

Komfort-Aliase für storage.get() und storage.set().

await CapytGame.submitScore({ score })

Benötigt game.score. Prüft den Score gegen resultValidation und aktualisiert lastScore/bestScore. Es wird dadurch noch kein Run abgeschlossen und kein Coin-Reward vergeben.

```
await CapytGame.submitScore({ score: 1240 });
```

await CapytGame.finish({ score, durationMs })

Benötigt game.score. Schließt den Host-generierten Run ab.

```
const result = await CapytGame.finish({  
  score: 1240,  
  durationMs: 34000  
});
```

Die Antwort enthält unter anderem:

```
runId  
score  
bestScore  
newHighScore  
coinsAwarded  
plays  
durationMs
```

8. Events und Nachrichten

Das SDK kapselt die postMessage-Details. Relevant für Debugging und API-Entwicklung sind unter anderem:

Game -> Host:

```
capyt.game.ready  
capyt.game.start  
capyt.game.getCapy
```

```
capyt.game.getAppInfo  
capyt.game.getTheme  
capyt.game.storage.get  
capyt.game.storage.set  
capyt.game.storage.remove  
capyt.game.submitScore  
capyt.game.finish
```

Host -> Game:

```
capyt.game.init  
capyt.game.response  
capyt.theme.changed  
capyt.game.lifecycle
```

Jede Request-Nachricht enthält `requestId`, `gameId` und `sessionId`. Der Host akzeptiert nur Nachrichten des aktuell geladenen iframe-Fensters und der aktiven Session. Das SDK akzeptiert Antworten und Lifecycle-/Theme-Nachrichten nur vom Parent-Fenster; Init-, Theme- und Lifecycle-Daten müssen zur aktuellen Game-/Session-ID passen.

9. Resultate, Highscores und Run-ID

Der Host speichert pro Game mindestens:

```
bestScore  
lastScore  
plays  
lastPlayedAt  
totalCoinsEarned
```

Zusätzlich werden verarbeitete `runIds` begrenzt gespeichert, damit derselbe Abschluss nicht mehrfach verarbeitet wird.

Eine doppelte `finish()` Verarbeitung derselben Run-ID:

- vergibt keine Coins erneut
- erhöht `plays` nicht erneut
- erzeugt keinen zweiten Reward

Temporäre Laufzeitdaten wie Pointer-Positionen, Gegner, Partikel oder Timer gehören nicht in den persistenten Game-State.

10. Coins und Reward Validation

Ein Game darf niemals direkt den Coin-Stand manipulieren.

Ein Game sendet nur sein Resultat. Der Host berechnet daraus den zulässigen Reward anhand von `games.json`.

Nicht zulässig:

```
coins += 100000;
```

und auch nicht vertrauenswürdig:

```
await CapytGame.finish({  
  score: 10,  
  durationMs: 5000,  
  coins: 100000  
});
```

Ein zusätzlich gesendetes `coins` Feld wird nicht zur Reward-Berechnung verwendet.

Game-Coins sind reine Capy-Game-Währung. Minigame-Starts, Scores, Highscores und Coin-Rewards erzeugen keine normale Finanzbuchung. Der Capy-Vorrat bleibt echtes Geld und ist davon

getrennt.

11. Finanz-App und Datenschutzgrenze

Minigames erhalten standardmäßig keinen Zugriff auf:

- Kontostände
- Budgets
- Buchungen
- Vorratseinzahlungen
- Bankdaten
- FP1-Payloads
- QR-Daten
- vollständige Finanz-State-Objekte

`app.read` liefert nur technische App-Informationen wie Version, Plattform, Locale, Online-Status und PWA-Status.

`finance.read.summary` ist für eine mögliche spätere API reserviert und in Game API 1 nicht freigeschaltet.

12. Game Storage und Persistenz

Der Host speichert Game-Daten im bestehenden Cappy-State unter einem games Bereich. Jedes Game ist nach `gameId` getrennt. Beispiel sinngemäß:

```
{
  "games": {
    "carrot-catch": {
      "bestScore": 42,
      "lastScore": 18,
      "plays": 7,
      "lastPlayedAt": "2026-08-13T12:00:00.000Z",
      "totalCoinsEarned": 21,
      "data": {
        "tutorialSeen": true
      }
    }
  }
}
```

Es wird keine zweite Datenbank für Minigames eingeführt. Die Struktur ist so gewählt, dass langfristige Daten später gezielt in FP1-Synchronisationen aufgenommen werden können. In 2.2.7a wird ein laufendes Game nicht zwischen Geräten synchronisiert.

13. Theme

Games müssen Light und Dark Mode unterstützen können. Das SDK setzt beim Handshake das aktuelle Theme am `<html>` Element:

```
:root {
  color-scheme: dark;
}

:root[data-theme="light"] {
  color-scheme: light;
}
```

Ein Theme-Wechsel während des Spiels wird über `capyt.theme.changed` übertragen und vom SDK angewendet.

Games dürfen eigene CSS-Variablen verwenden. Sie sollen globale Cappyt-CSS-Dateien nicht manipulieren.

14. Offline/PWA

Der Service Worker cached immer:

- games.json
- Game-Hub CSS
- Game Hub
- Game Loader
- Game Bridge
- Reward Service
- Game Storage
- Game SDK

Zusätzlich liest der Service Worker während der Installation games.json. Für jedes aktivierte available oder experimental Game werden automatisch gecacht:

- entry
- alle Einträge aus offlineAssets

Beim Hinzufügen eines neuen aktivierten Games müssen daher alle für Offline-Betrieb nötigen lokalen Dateien in offlineAssets eingetragen werden. Assets dürfen nur im eigenen Projektordner liegen.

Wenn Capyt offline ist, zeigt der Hub gecachte Games weiterhin an. Ein nicht gecachtes Game wird als offline nicht verfügbar behandelt, statt den Hub abstürzen zu lassen.

Nach Änderungen an produktiven PWA-Dateien muss die Capyt-Version bzw. der Service-Worker-Build so aktualisiert werden, dass ein neuer Cache entsteht.

15. Minimales vollständiges Beispiel

index.html:

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <link rel="stylesheet" href="./game.css">
</head>
<body>
  <button id="start">Start</button>
  <button id="point" disabled>+1</button>
  <button id="finish" disabled>Fertig</button>
  <strong id="score">0</strong>

  <script src="../../js/game-sdk.js"></script>
  <script src="./game.js"></script>
</body>
</html>
```

game.js:

```
let score = 0;
let startedAt = 0;

const init = await CapytGame.ready();
console.log('Capy:', init.capy?.name);

document.querySelector('#start').onclick = async () => {
  await CapytGame.start();
  startedAt = performance.now();
  document.querySelector('#point').disabled = false;
  document.querySelector('#finish').disabled = false;
};

document.querySelector('#point').onclick = () => {
  score += 1;
```

```

document.querySelector('#score').textContent = score;
};

document.querySelector('#finish').onclick = async () => {
  const result = await CapytGame.finish({
    score,
    durationMs: Math.round(performance.now() - startedAt)
  });
  console.log(result);
};

```

Passender Registry-Eintrag:

```

{
  "id": "mein-game",
  "name": "Mein Game",
  "description": "Kurzes Beispielspiel.",
  "status": "available",
  "enabled": true,
  "version": "1.0.0",
  "entry": "./projects/mein-game/index.html",
  "icon": "MG",
  "image": "",
  "orientation": "portrait",
  "order": 100,
  "tags": ["quick"],
  "rewards": {
    "enabled": true,
    "currency": "coins",
    "maxCoinsPerRun": 5,
    "dailyRewardLimit": 20,
    "cooldownMs": 0,
    "strategy": { "type": "score", "scorePerCoin": 10 }
  },
  "resultValidation": {
    "minScore": 0,
    "maxScore": 1000,
    "minDurationMs": 0,
    "maxDurationMs": 600000
  },
  "capabilities": [
    "copy.read",
    "game.score",
    "coins.reward",
    "theme.read"
  ],
  "offlineAssets": [
    "./projects/mein-game/game.js",
    "./projects/mein-game/game.css"
  ]
}

```

16. Tests und Checkliste

Vor dem Aktivieren eines neuen Games prüfen:

- Registry lädt ohne Fehler.
- Game-ID ist eindeutig.
- Entry-Path bleibt unter `copy/games/projects/`.
- `coming_soon` Games sind nicht startbar.
- Nur notwendige Capabilities sind eingetragen.
- Score- und Dauergrenzen sind plausibel.
- `maxCoinsPerRun` und `dailyRewardLimit` sind gesetzt.
- Wiederholtes `finish()` dupliziert keinen Reward.

- Game Storage bleibt im eigenen Namespace.
- Light und Dark Mode funktionieren.
- Game funktioniert offline nach PWA-Installation.
- Alle benötigten Assets stehen in `offlineAssets`.
- Das Game erzeugt keine Finanzbuchung.

Repository-Tests:

```
npm test  
npm run check
```

17. Referenzimplementierungen

- `projects/carrot-catch/`: kleines produktives Referenz-Game
- `projects/bridge-demo/`: deaktiviertes internes Bridge-Demo
- `projects/_template/`: Startpunkt für neue Games

Der Game-Hub darf von keiner dieser Referenzimplementierungen technisch abhängig sein. Die Plattform arbeitet ausschließlich über Registry, Loader, Bridge und die standardisierte API.